

CPT204 Data Structures and Algorithms - 22-23F

Part B. Coding Questions

Question 1: Coding 1

Complete the overridden method void addFirst(T item). It is a method of the class DelimitedARDeque, a subclass of an iterable ARDeque. It adds an item to the front of the object of DelimitedARDeque, but it will throw an IllegalArgumentException if the item already occurs in it more than once.

Test case:

```
DelimitedARDeque<String> dd = new DelimitedARDeque<>();
dd.addFirst("a");
dd.addFirst("b");
try {
    dd.addFirst("a");
    dd.printDeque();
} catch (IllegalArgumentException e) {
} -> AddF
```

code:

```
import java.util.Iterator;
import java.util.NoSuchElementException;

public class ARDeque<T> implements Iterable<T>, Cloneable {
    private T[] items;
    private int front;
    private int size;

    @SuppressWarnings("unchecked")
    public ARDeque() {
        items = (T[]) new Object[8];
        front = 0;
        size = 0;
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }
}
```

```
public void addFirst(T item) {
    ensureCapacity();

    front = Math.floorMod(front - 1, items.length);
    items[front] = item;
    size++;
}

public boolean contains(T item) {
    for (T i : this) {
        if (i.equals(item)) {
            return true;
        }
    }
    return false;
}

public void addLast(T item) {
    ensureCapacity();

    int lastIndex = Math.floorMod(front + size, items.length);
    items[lastIndex] = item;
    size++;
}

public T removeFirst() {
    if (isEmpty()) {
        throw new NoSuchElementException("Deque is empty");
    }

    T item = items[front];
    items[front] = null;
    front = Math.floorMod(front + 1, items.length);
    size--;

    shrinkIfNeeded();
    return item;
}

public T removeLast() {
    if (isEmpty()) {
        throw new NoSuchElementException("Deque is empty");
    }

    int lastIndex = Math.floorMod(front + size - 1, items.length);
    T item = items[lastIndex];
    items[lastIndex] = null;
    size--;

    shrinkIfNeeded();
    return item;
}

public T getFirst() {
    if (isEmpty()) {
        throw new NoSuchElementException("Deque is empty");
    }
}
```

```

    }

    return items[front];
}

public T getLast() {
    if (isEmpty()) {
        throw new NoSuchElementException("Deque is empty");
    }

    int lastIndex = Math.floorMod(front + size - 1, items.length);
    return items[lastIndex];
}

public void printDeque() {
    for (T item : this) {
        System.out.print(item + " ");
    }
    System.out.println();
}

private void ensureCapacity() {
    if (size == items.length) {
        resize(items.length * 2);
    }
}

private void shrinkIfNeeded() {
    if (items.length > 8 && size <= items.length / 4) {
        resize(items.length / 2);
    }
}

@SuppressWarnings("unchecked")
private void resize(int newCapacity) {
    T[] newItems = (T[]) new Object[newCapacity];

    for (int i = 0; i < size; i++) {
        newItems[i] = items[Math.floorMod(front + i, items.length)];
    }

    items = newItems;
    front = 0;
}

@Override
public Iterator<T> iterator() {
    return new Iterator<T>() {
        private int index = 0;

        @Override
        public boolean hasNext() {
            return index < size;
        }

        @Override
        public T next() {

```

```

        if (!hasNext()) {
            throw new NoSuchElementException();
        }

        T item = items[Math.floorMod(front + index, items.length)];
        index++;
        return item;
    }
}

@Override
public ARDeque<T> clone() {
    ARDeque<T> cloned = new ARDeque<T>();
    for (T item : this) {
        cloned.addLast(item);
    }
    return cloned;
}
}

```

```

public class DelimitedARDeque<T> extends ARDeque<T> {
    @Override
    public void addFirst(T item) throws IllegalArgumentException {
        // check whether the item has been in the deque
        if (super.contains(item))
            throw new IllegalArgumentException("Item " + item.toString() + " already
            ↪ exists in the deque");
        super.addFirst(item);
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        DelimitedARDeque<String> dd = new DelimitedARDeque<>();
        try {
            dd.addFirst("apple");
            dd.addLast("banana");
            dd.addFirst("cherry");
            dd.addFirst("apple");
        } catch (IllegalArgumentException e) {
            System.out.println("IllegalArgumentException: " + e.getMessage());
        }
        dd.printDeque();
    }
}

```

Question 2: Coding 2

Complete the method boolean equals(Object that) of an iterable ARDeque that overrides the one in Object. To be the same, they must be of the same size, and must share at most 1 different items at the same position. Suppose [] is an ARDeque. Then: [1, 2, 3].equals([1, 2, 5]) -> true [1, 3, 2].equals([1, 2, 3]) -> false

```
import java.util.Iterator;
import java.util.NoSuchElementException;
import java.util.Objects;
import java.util.stream.IntStream;

public class ARDeque<T> implements Iterable<T>, Cloneable {
    private T[] items;
    private int front;
    private int size;

    @SuppressWarnings("unchecked")
    public ARDeque() {
        items = (T[]) new Object[8];
        front = 0;
        size = 0;
    }

    public ARDeque(T[] initialItems) {
        items = (T[]) new Object[Math.max(8, initialItems.length * 2)];
        System.arraycopy(initialItems, 0, items, 0, initialItems.length);
        front = 0;
        size = initialItems.length;
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public void addFirst(T item) {
        ensureCapacity();

        front = Math.floorMod(front - 1, items.length);
        items[front] = item;
        size++;
    }

    public boolean contains(T item) {
        for (T i : this) {
            if (i.equals(item)) {
                return true;
            }
        }
        return false;
    }

    public void addLast(T item) {
        ensureCapacity();

        int lastIndex = Math.floorMod(front + size, items.length);
        items[lastIndex] = item;
        size++;
    }
}
```

```
public T removeFirst() {
    if (isEmpty()) {
        throw new NoSuchElementException("Deque is empty");
    }

    T item = items[front];
    items[front] = null;
    front = Math.floorMod(front + 1, items.length);
    size--;

    shrinkIfNeeded();
    return item;
}

public T removeLast() {
    if (isEmpty()) {
        throw new NoSuchElementException("Deque is empty");
    }

    int lastIndex = Math.floorMod(front + size - 1, items.length);
    T item = items[lastIndex];
    items[lastIndex] = null;
    size--;

    shrinkIfNeeded();
    return item;
}

public T getFirst() {
    if (isEmpty()) {
        throw new NoSuchElementException("Deque is empty");
    }

    return items[front];
}

public T getLast() {
    if (isEmpty()) {
        throw new NoSuchElementException("Deque is empty");
    }

    int lastIndex = Math.floorMod(front + size - 1, items.length);
    return items[lastIndex];
}

public void printDeque() {
    for (T item : this) {
        System.out.print(item + " ");
    }
    System.out.println();
}

public String toString() {
    StringBuilder sb = new StringBuilder();
    sb.append("[ ");
    for (var item : this) {
```

```

        sb.append(item).append(" ");
    }
    sb.append(" ]");
    return sb.toString();
}

private void ensureCapacity() {
    if (size == items.length) {
        resize(items.length * 2);
    }
}

private void shrinkIfNeeded() {
    if (items.length > 8 && size <= items.length / 4) {
        resize(items.length / 2);
    }
}

@SuppressWarnings("unchecked")
private void resize(int newCapacity) {
    T[] newItems = (T[]) new Object[newCapacity];

    for (int i = 0; i < size; i++) {
        newItems[i] = items[Math.floorMod(front + i, items.length)];
    }

    items = newItems;
    front = 0;
}

@Override
public Iterator<T> iterator() {
    return new Iterator<T>() {
        private int index = 0;

        @Override
        public boolean hasNext() {
            return index < size;
        }

        @Override
        public T next() {
            if (!hasNext()) {
                throw new NoSuchElementException();
            }

            T item = items[Math.floorMod(front + index, items.length)];
            index++;
            return item;
        }
    };
}

@Override
public boolean equals(Object that) {
    if (this == that) {
        return true;
    }

```

```

        // if same object (reference type), then they are equal
    }

    // test if object is subclass or same class of ARDeque
    if (!(that instanceof ARDeque)) {
        return false;
    }

    // then compare the size of two deques and compare the item in the deque
    ARDeque<?> other = (ARDeque<?>) that;
    if (this.size != other.size) {
        return false;
    }

    // if two queues only have one or less difference in position, then they are
    // equal
    return IntStream.range(0, this.size)
        .filter(i -> !Objects.equals(
            this.items[Math.floorMod(this.front + i, this.items.length)],
            other.items[Math.floorMod(other.front + i,
↪ other.items.length]]))
        .limit(2)
        .count() <= 1;
    }

    @Override
    public ARDeque<T> clone() {
        ARDeque<T> cloned = new ARDeque<T>();
        for (T item : this) {
            cloned.addLast(item);
        }
        return cloned;
    }
}

```

```

public class DelimitedARDeque<T> extends ARDeque<T> {
    DelimitedARDeque(T[] initialItems) {
        super(initialItems);
    }

    DelimitedARDeque() {
        super();
    }

    @Override
    public void addFirst(T item) throws IllegalArgumentException {
        // check whether the item has been in the deque
        if (super.contains(item))
            throw new IllegalArgumentException("Item " + item.toString() + " already
↪ exists in the deque");
        super.addFirst(item);
    }

    public static void main(String[] args) {
        DelimitedARDeque<String> dd = new DelimitedARDeque<>();
        try {

```



```
        dd.addFirst("apple");
        dd.addLast("banana");
        dd.addFirst("cherry");
        dd.addFirst("apple");
    } catch (IllegalArgumentException e) {
        System.out.println("IllegalArgumentException: " + e.getMessage());
    }
    dd.printDeque();
}
```

```
DelimitedARDeque<Integer> dd1 = new DelimitedARDeque<>(new Integer[] { 1, 2, 3 });
DelimitedARDeque<Integer> dd2 = new DelimitedARDeque<>(new Integer[] { 1, 4, 3 });
DelimitedARDeque<Integer> dd3 = new DelimitedARDeque<>(new Integer[] { 2, 4, 3 });
System.out.println("dd1: " + dd1.toString());
System.out.println("dd2: " + dd2);
System.out.println("dd3: " + dd3);
System.out.println("dd1 equals dd2: " + dd1.equals(dd2));
System.out.println("dd1 equals dd3: " + dd1.equals(dd3));
```